LENGUAJE DE PROGRAMACIÓN II / SESIÓN 13

Optimización y preparación para producción de PharmaSoft

Actividad autónoma · Reto 02 · Semestre 2026-2

Jatniel Jair Suarez Vigo

Facultad de Ingeniería y Arquitectura · Ingeniería de Sistemas
Docente: Benjamin David Reyna Barreto · Mediciones: 8 de noviembre de 2026

Resultados principales

Prueba	Resultado observado
Entrega inicial	318.190 → 306.910 bytes de build (−3,54 %).
Navegación	Sin precarga se transfirieron 52.877 bytes menos antes de abrir Ventas.
Caché Redis	19 de 20 aciertos por caché; seis verificaciones de la matriz correctas.
Despliegue	Texto nuevo en Productos con una recarga normal y el mismo Nginx.
Observabilidad	Venta 161: cuatro líneas con el mismo X-Request-Id.

Entorno y alcance

Se usó una compilación Angular de producción servida por Nginx en **localhost:8081**. PharmaBackend corrió en el puerto 8080 con perfil prod, Oracle persistió los datos y Redis funcionó como caché temporal. La prueba de TTL usó un proceso aparte y la base 1 de Redis.

Los resultados corresponden al laboratorio y a los casos ejecutados. Se conservaron capturas y registros JSON junto al código del frontend. La publicación y el requisito de tres días se detallan en la parte F.

Parte	Contenido
A–B	Paquete inicial, precarga, política de caché y despliegue simulado.
C–D	Aciertos, TTL, invalidación, trazas y monitoreo.
E–F	Despliegue en tres locales y estado de entrega.

PARTE A / 4 PUNTOS

Paquete inicial y carga diferida

Dos compilaciones de producción con el mismo comando: ng build --stats-json.

#	Módulo de entrada	Origen	Bytes aportados
1	_debug_node-chunk.mjs	Angular Core · ejecución y renderizado	139.534
2	_router-chunk.mjs	Angular Router · navegación y guardas	86.572
3	_pending_tasks-chunk.mjs	Angular Core · tareas pendientes	35.115
4	_module-chunk.mjs	Angular Common · cliente HTTP	25.137
5	_common_module-chunk.mjs	Angular Common · utilidades comunes	22.229

Los cinco módulos son entradas de Angular identificadas en el metafile de esbuild. El nombre de un archivo `chunk-*.js` identifica un resultado de compilación que puede reunir varios módulos. Los bytes aportados son estimaciones del analizador; el total real se toma del build.



A-analyzer.png · Metafile de la compilación anterior en esbuild Bundle Analyzer.

Cambio aplicado y medido

Indicador	Antes	Después
Initial total bruto	318.190 B	306.910 B
Transferencia estimada del build	89,63 kB	86,58 kB
Chunks JS diferidos	23	29

En **app.routes.ts** pasé el layout y la pantalla de acceso denegado a `loadComponent`. En **ventas.routes.ts** separé lista, formulario y detalle para descargar cada pantalla al abrirla. El login deja de cargar el layout de una sesión y entrar a Ventas no trae el formulario y el detalle por adelantado.

PARTE A / CONTINUACIÓN

Comparación de precarga

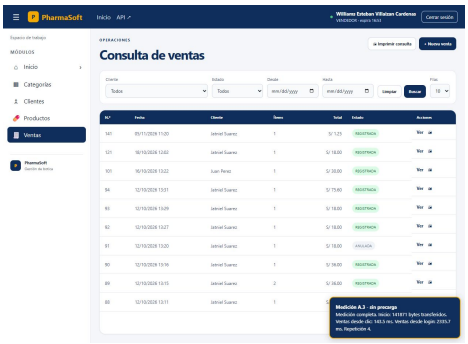
Tres ejecuciones por variante con el mismo backend, navegador de 1280 × 720, gzip y caché HTTP deshabilitada por la configuración del banco de pruebas.

Mediana de 3 corridas	Sin precarga	Con precarga
Bytes hasta clic en Ventas	141.871 B	194.748 B
Bytes JS hasta el clic	139.101 B	191.977 B
Clic → título de Ventas	43,0 ms	45,7 ms
Clic → primera fila	152,6 ms	104,9 ms
Login aceptado → primera fila	2.335,7 ms	2.280,0 ms

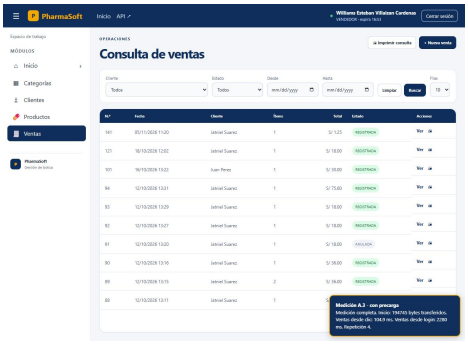
Inicio de sesión con el mismo rol y espera de dos segundos en Inicio. Se midió hasta ver una fila, además del título, para incluir la carga de los datos. El intervalo desde el login incluye la espera fijada para ambos escenarios.

Decisión: conservar la carga diferida sin precarga global

Precargar todo aumentó los bytes previos a Ventas en **37,27 %** y adelantó la primera fila **47,7 ms** desde el clic, sin mejorar la aparición del título en la mediana. En los locales con conexión más lenta conviene descargar cada pantalla al usarla. Si Ventas concentra el trabajo, medir una precarga selectiva antes de adoptarla.



A-sin-precarga.png · Medición en el navegador con la opción predeterminada.



A-con-precarga.png · Misma prueba con withPreloading(PreloadAllModules).

PARTE B / 3 PUNTOS

Caché HTTP y actualización

Nginx en 8081. Se verificó Cache-Control con curl -I y la transferencia con Resource Timing del navegador durante una recarga normal.

Recurso	Cache-Control	Origen en la recarga	OK
/index.html	no-cache	Revalidación: 300 B observados; 304 con If-None-Match en curl.	Sí
main-OGNHT4Q7.js	public, max-age=31536000, immutable	Caché local: 0 B.	Sí
chunk-807RgT9E.js	public, max-age=31536000, immutable	Caché local: 0 B al volver a Ventas.	Sí
styles-E6WR7BPR.css	public, max-age=31536000, immutable	Caché local: 0 B.	Sí
/api/v1/categorias	no-cache, no-store, max-age=0, must-revalidate	Red: 999 B, HTTP 200.	Sí

Resource Timing permite confirmar la reutilización local sin distinguir memoria de disco. El 304 se verificó con una solicitud condicional separada; el navegador expone 200 en la entrada de navegación. No se limpió la caché entre versiones.

Simulación de despliegue

Archivo	Antes	Después	Recarga posterior
Entrada	main-OGNHT4Q7.js	main-RW7DF4L2.js	2.429 B
Lista de Productos	chunk-CU9zIPZQ.js	chunk-DuCnzr9e.js	4.459 B
Lista de Ventas	chunk-807RgT9E.js	chunk-CvPknq0a.js	Fuera de la ruta probada
Estilos	styles-E6WR7BPR.css	styles-E6WR7BPR.css	0 B

El contenedor Nginx conservó su ID (ebcff7ba9fcf) y hora de arranque. El HTML revalidado referencia nombres nuevos para el código que cambió. El CSS conservó su hash y se reutilizó. El nuevo chunk de Ventas aparece en el manifiesto; la recarga de Productos no lo solicita.

Antes: texto original



Después: texto nuevo



B-antes-despliegue.png y B-despues-despliegue.png · Recortes de la misma vista de Productos.

El API contiene datos autenticados y variables como el stock. Spring Security agrega **no-cache, no-store, max-age=0, must-revalidate**; así se evita que el navegador reutilice respuestas anteriores o conserve copias de datos de sesión. La caché de Redis actúa en el servidor y se invalida al escribir.

PARTE C / 5 PUNTOS

Aciertos, tiempos y vigencia

Pruebas con la aplicación recién iniciada y dos claves de prueba retiradas al comenzar cada serie.

Consulta	GET	Hit / miss	Tasa	Primer fallo	Media 19 hit
GET /categorias	20	19 / 1	95 %	37,785 ms	23,104 ms
GET /productos · página 0	20	19 / 1	95 %	43,226 ms	25,355 ms

Para cada caché se restó el contador anterior al posterior en `/actuator/metrics/cache.gets` con las etiquetas `name` y `result`. Tasa: 19 / (19 + 1) = 95 %. Los tiempos se midieron desde el cliente HTTP con un reloj monotónico, incluyendo autorización y transferencia.

El primer GET consulta Oracle y escribe Redis. Los 19 siguientes leen la copia. La diferencia no equivale a un benchmark general: hay un único fallo en frío por serie y la red es local.

TTL temporal de un minuto

Tiempo de muestreo	redis-cli TTL	redis-cli EXISTS
0,0 s	60	1
11,4 s	48	1
22,7 s	37	1
34,0 s	25	1
45,4 s	14	1
56,5 s	3	1
67,9 s	-2	0

La instancia temporal del puerto 8082 escribió en Redis DB 1 con **1m**. La muestra final devolvió TTL −2 y EXISTS 0, que confirman la expiración. Se detuvo esa instancia. En el backend normal (8080, DB 0), un GET repobló la clave con **600 segundos**.

```
Comandos de verificación en la sesión TTL:
redis-cli -n 1 TTL "categorias::todas"
redis-cli -n 1 EXISTS "categorias::todas"
redis-cli -n 0 TTL "categorias::todas" → 600
```

Fuentes: C-aciertos-20x2.json · C-ttl-1-minuto.json · C-ttl-restaurado.json.

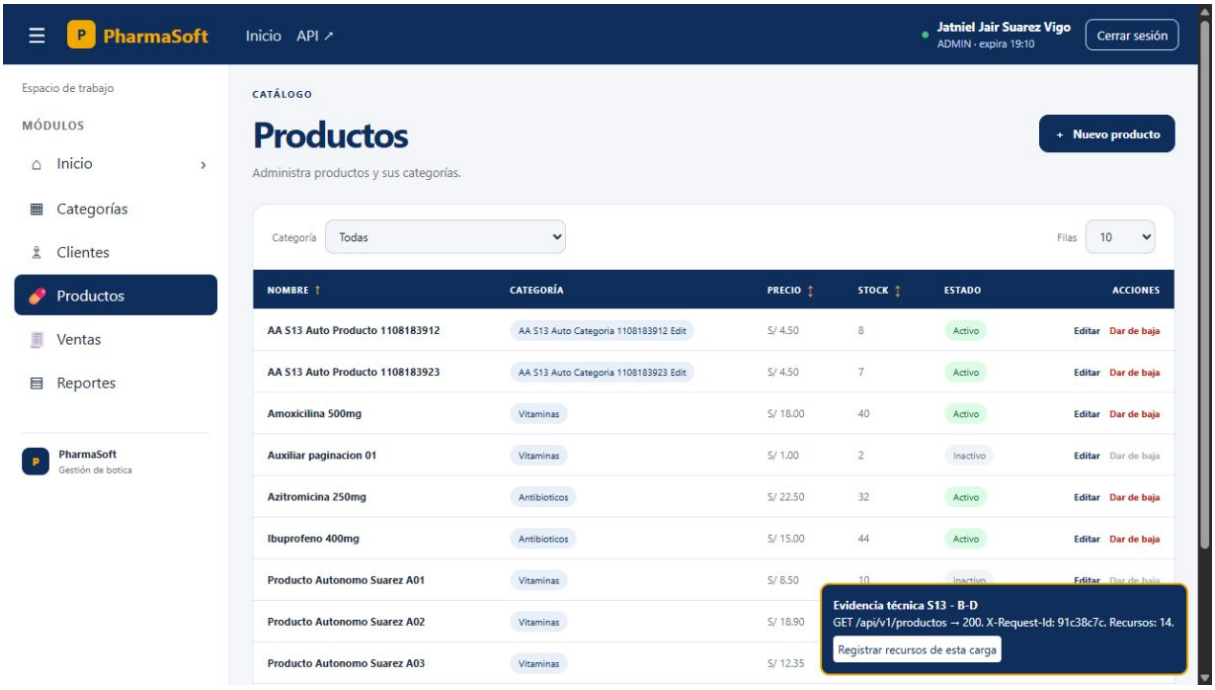
PARTE C / CONTINUACIÓN

Matriz de invalidación

Se probaron lecturas después de cada escritura con datos creados para el ensayo. Oracle permaneció activo durante la caída de Redis.

ID	Operación	Esperado	Obtenido
C-01	Crear categoría	Aparece en lista	Visible, ID 163; clave invalidada.
C-02	Renombrar categoría	Lista de productos muestra el nombre nuevo	La categoría del producto 122 muestra el sufijo Edit.
C-03	Cambiar precio	Nuevo precio en detalle y lista	S/ 3,25 → S/ 4,50 en ambas vistas.
C-04	Registrar venta desde SPA	Stock descontado	8 → 7 tras vender una unidad; venta 161.
C-05	Eliminar categoría vacía	Desaparece de la lista	ID 164 ausente; clave invalidada.
C-06	Detener Redis	Productos funciona; health DOWN	Productos HTTP 200; health HTTP 503, Redis DOWN.

C-01 a C-05 verificaron la respuesta inmediatamente posterior a la escritura. No se halló un defecto de invalidación en estos casos. Las escrituras de categorías limpian la caché de categorías y, cuando corresponde, la de productos; editar un producto o registrar una venta limpia productos.



C-06-productos-spa.png · La SPA muestra el producto de prueba con precio S/ 4,50 y stock 7, mientras Redis está detenido.

La comprobación directa de Productos con Redis apagado tomó 581,9 ms. El manejador de errores de caché permitió consultar Oracle. El retorno a UP se comprobó después de iniciar Redis.

PARTE D / 4 PUNTOS

Trazabilidad de la venta y los errores

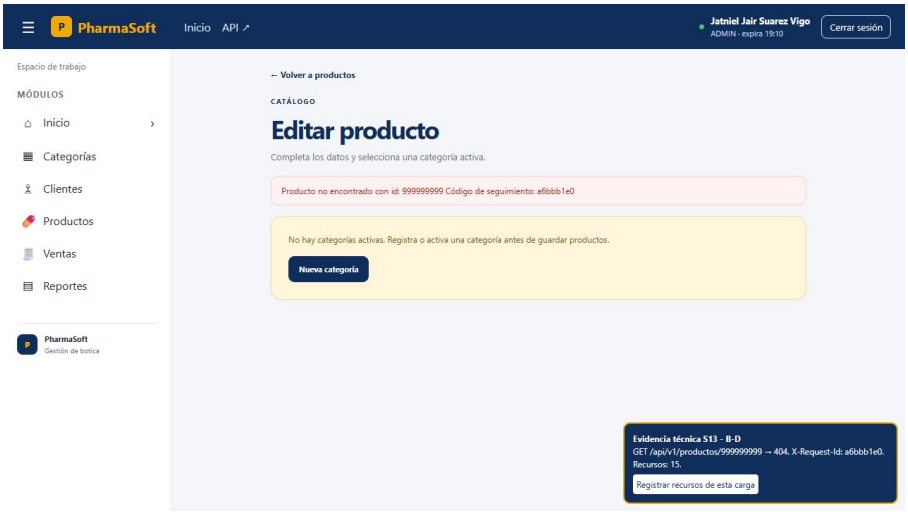
La SPA registró una venta de una unidad del producto de prueba; se relacionó la respuesta HTTP con las líneas del log de producción.

```
POST /api/v1/ventas → 201 Created
X-Request-Id: e6438c16 | Venta: 161
Filtro de log: Select-String -SimpleMatch "e6438c16"
```

Hora / mismo identificador	Mensaje de log	Interpretación
18:41:17,297 [e6438c16]	Inicio de registro de venta. clienteId=22, cantidadDetalles=1	Inicio: una línea de venta para el cliente seleccionado.
18:41:17,303 [e6438c16]	Procesando producto. productoId=122, cantidadSolicitada=1	Proceso: valida la cantidad del producto de prueba.
18:41:17,374 [e6438c16]	Venta registrada con éxito. ventaId=161, total=4.5	Persistencia: confirma la venta y el total de S/ 4,50.
18:41:17,395 [e6438c16]	POST /api/v1/ventas -> 201 (173 ms)	Salida HTTP: el filtro anota estado 201 y duración.

Identificador visible en la SPA

Agregué **requestId** a **ErrorResponseDTO**. **GlobalExceptionHandler** lee el valor de `MDC.get("requestId")`; las respuestas 401/403 del filtro de seguridad usan el mismo campo. En Angular, **http-error.ts** lo agrega al mensaje cuando existe. El usuario puede copiarlo y soporte puede filtrar las líneas de esa petición.



D-error-spa.png · Error 404 real. Cabecera, JSON y alerta muestran a6bbb1e0.

Registros: D-venta-post.json, D-venta-traza.json y D-error-spa.json. Se guardaron los campos de trazabilidad necesarios, sin Authorization ni cuerpo del login.

PARTE D / CONTINUACIÓN

Salud, métricas y auditoría

Se midió el estado global y los componentes al detener Redis y después de restaurarlo.

Redis	Health público	Health con token admin	Interpretación
Activo	200 UP; sin detalles	200 UP; Oracle UP y Redis UP	Servicios locales operativos.
Detenido	503 DOWN; sin detalles	503 DOWN; Oracle UP y Redis DOWN	Caché caída; el listado sigue sirviéndose desde Oracle.
Restaurado	200 UP	200 UP; ambos componentes UP	Terminó la prueba de caída.



D-health-down.png · /actuator/health sin token durante la caída (recorte).



D-health-up.png · /actuator/health sin token tras restaurar Redis (recorte).

Métrica http.server.requests · uri:/api/v1/ventas

COUNT	TOTAL_TIME	MAX	Etiquetas observadas
1	0,173444 s	0,173444 s	POST · 201 · SUCCESS

La métrica representa un POST exitoso en el proceso medido: promedio = total / conteo = 173,444 ms. Con una sola muestra el máximo coincide con el promedio y no describe el rendimiento bajo carga. Vendedor recibió 403 al consultar métricas.

Auditoría de secretos

En 72.252 bytes del log se buscaron los patrones password, Bearer y secret, la clave JWT, tokens y contraseña de base de datos, y 6 DNI de clientes conocidos: **0 coincidencias en la muestra**. Como prevención, el filtro de JWT registra la clase de excepción en vez del mensaje, que podría incluir contenido del token.

Fuentes: D-auditoria-y-metricas.json, D-health-up.json y C-06-redis-down.json. Suite Maven: 15 pruebas, 0 fallos y 0 errores. Build Angular: correcto, con advertencia de 57 B del CSS de Venta.

PARTES E Y F / 4 PUNTOS

Despliegue y entrega

Los dos README incluyen arquitectura, variables sin valores reales, verificación, reversión y una lista de buenas prácticas.

```
Navegadores de los 3 locales
→ Nginx (8081 local; HTTPS 443 propuesto)
→ PharmaBackend (8080 privado) para /api/
→ Oracle (1521 interno, 1522 host local)
→ Redis (6379 privado, TTL 10m)
```

Prioridad	Acción para los tres locales
1 · Acceso	Publicar una sola URL HTTPS con renovación y mantener backend, Oracle y Redis en red privada.
2 · Continuidad	Respaldar Oracle fuera del servidor y probar restauración y reversión en un local antes del despliegue general.
3 · Operación	Compartir backend y Redis privado, alertar por health DOWN y 5xx, y medir latencia desde los tres locales.

Conservar cada JAR y carpeta de frontend por versión. Nginx monta la versión con **SPA_DIST**; para revertir se monta la anterior. El JAR anterior solo se reinicia si es compatible con el esquema migrado. Verificar health, login y Productos al terminar.

Versionamiento y entrega

Requisito	Estado
Ramas de trabajo	feature/optimizacion-produccion-suarez en backend y frontend.
Tres días distintos	Trabajo S13 registrado el 5 y el 8 de noviembre. Queda un tercer día real por cumplir en cada repositorio.
PDF	Backend: docs/informe-s13.pdf. Copia para el aula: Suarez_LP2_S13_Autonoma.pdf.
Solicitudes hacia develop	Pendientes de publicación. Frontend tiene origin/develop; en backend no se encontró esa rama remota en la revisión local.

Fuentes y material de sustentación

Guía práctica LP II · sesión 13, pasos 2 a 13. Actividad autónoma LP II · sesión 13, partes A a F. Metafile de esbuild, build Angular, capturas del navegador, cabeceras de curl, salida de redis-cli, logs y métricas Actuator del laboratorio.

Para repetir las pruebas: docs/s13-autonoma.md, tools/benchmark/README.md y tools/s13_autonoma.py. Evidencias por parte: docs/evidencias-s13-autonoma en el frontend.